JAVA LANGUAGE REFRESHER

CHAPTER 02 · VARIABLES TO LOOPS

Deep refresher · 8–9/10 target · Java 17+

> COLOR LEGEND purple = keywords and operators blue = numbers orange = strings teal = comments

38 illustrated Java examples with verified output beneath each. Download includes 8 combined, topic-based Java programs for IntelliJ IDEA.

Focus: variables, all primitive ranges, numeric conversions, String, operators, decisions, and loops. Arrays and OOP explanations are saved for later chapters.

YOUR LEARNING MAP

1. Variables, identifiers and scope
2. Primitive data types and ranges
3. Conversions, promotion and casting
4. Strings in depth
5. Operators and expression evaluation
6. Decisions: if, else and switch
7. Loops: repetition and control
8. Synthesis: predicting output and fixing bugs
9. Mastery checks and reference

1. Variables, identifiers and scope

A variable is a named location for a value. Java checks its type at compile time. Declarations tell Java what type a variable holds; assignments give it a value.

> Core mental model Declaration: int count; Assignment: count = 4; Initialization: int count = 4; Reassignment changes a variable, not its type. Local variables require assignment before reading; fields receive defaults (covered in OOP).

Declare, initialize and reassign · Example 01

Notice that Java is statically typed; type is not inferred again when the value changes.

```
/**
 * CHAPTER 2 | Declare, initialize and reassign
 * WHY IT MATTERS: Notice that Java is statically typed; type is not inferred again
 * when the value changes.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex01Variables {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int age = 46;                   // int stores whole numbers
        double price = 19.95;           // decimal values
        boolean active = true;          // true or false
        char grade = 'A';               // one UTF-16 code unit
        // STEP 2: Compare the output with your prediction.
        System.out.println(age + " | " + price + " | " + active + " | " + grade);
        age = 47;                       // reassignment, still int
        System.out.println("Next age: " + age);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
46 | 19.95 | true | A
Next age: 47
```

Identifiers, naming and final · Example 02

Java names are case-sensitive. Prefer lowerCamelCase for variables and UPPER_SNAKE_CASE for constants.

```
/**
 * CHAPTER 2 | Identifiers, naming and final
 * WHY IT MATTERS: Java names are case-sensitive. Prefer lowerCamelCase for variables
 * and UPPER_SNAKE_CASE for constants.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex02NamesAndConstants {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int customerCount = 20;         // descriptive lowerCamelCase
        int customerCOUNT = 3;          // a DIFFERENT name
        final double SALES_TAX_RATE = 0.0825;
        double subtotal = 100;
        // STEP 2: Compare the output with your prediction.
        System.out.println(customerCount + customerCOUNT);
        System.out.println(subtotal * SALES_TAX_RATE);
        // SALES_TAX_RATE = 0.09;       // compile error: final variable
```

```
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
23
8.25
```

Block scope and shadowing rules · Example 03

Local variables exist only within the block where declared. You cannot redeclare the same local name in an overlapping block.

```
/**
 * CHAPTER 2 | Block scope and shadowing rules
 * WHY IT MATTERS: Local variables exist only within the block where declared. You
 * cannot redeclare the same local name in an overlapping block.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex03Scope {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int outer = 10;
        if (outer > 0) {
            int inner = 20;            // only visible inside this block
            // STEP 2: Compare the output with your prediction.
        System.out.println(outer + inner);
        }
        // System.out.println(inner);   // compile error: out of scope
        System.out.println(outer);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
30
10
```

Local variable type inference with var · Example 04

Since Java 10, var asks the compiler to infer the type from the initializer. It is still statically typed; it is not a dynamic variable.

```
/**
 * CHAPTER 2 | Local variable type inference with var
 * WHY IT MATTERS: Since Java 10, var asks the compiler to infer the type from the
 * initializer. It is still statically typed; it is not a dynamic variable.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex04VarInference {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        var count = 10;                 // inferred int
        var message = "Hello";         // inferred String
        var rate = 2.5;                 // inferred double
        // count = "ten";               // compile error
        // var missing;                  // compile error, no initializer
        // STEP 2: Compare the output with your prediction.
        System.out.println(message + " " + count * rate);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Hello 25.0
```

> **Identifier rules** Names can contain letters, digits, underscore, and dollar sign, but cannot start with a digit. Keywords (e.g., class, int, if) are disallowed; _ alone is reserved. Avoid dollar signs and confusing Unicode identifiers in ordinary code.

2. Primitive data types and ranges

Java has exactly eight primitive types: byte, short, int, long, float, double, char and boolean. String is NOT a primitive; it is a reference type.

Type	Size	Range / meaning	Literal hint
byte	8 bits	-128 to 127	(byte) 12
short	16 bits	-32,768 to 32,767	(short) 12
int	32 bits	-2,147,483,648 to 2,147,483,647	123
long	64 bits	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	123L
float	32 bits	~6-7 significant decimal digits; finite magnitude ~1.4E-45 to 3.4E38	1.2F
double	64 bits	~15-16 significant decimal digits; finite magnitude ~4.9E-324 to 1.8E308	1.2 (default)
char	16 bits	0 to 65,535 UTF-16 code units (\u0000 to \uFFFF)	'A'
boolean	JVM-dependent	true or false; no specified storage size	true

> **Precision is not decimal accuracy** float and double store binary floating-point approximations. double is the usual choice for non-financial scientific/measurement calculations. For exact decimal money, use BigDecimal (introduced briefly here; later chapter can cover APIs).

Print official integer and character ranges · Example 05

MIN_VALUE and MAX_VALUE are the built-in constants for numeric primitive wrapper types.

```
/**
 * CHAPTER 2 | Print official integer and character ranges
 * WHY IT MATTERS: MIN_VALUE and MAX_VALUE are the built-in constants for numeric
 * primitive wrapper types.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex05PrimitiveRanges {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        // STEP 2: Compare the output with your prediction.
        System.out.println("byte:   " + Byte.MIN_VALUE + " .. " + Byte.MAX_VALUE);
        System.out.println("short:  " + Short.MIN_VALUE + " .. " + Short.MAX_VALUE);
        System.out.println("int:    " + Integer.MIN_VALUE + " .. " + Integer.MAX_VALUE);
```

```
        System.out.println("long:   " + Long.MIN_VALUE + " .. " + Long.MAX_VALUE);
        System.out.println("char numeric: " + (int) Character.MIN_VALUE + " .. " + (int) Character.MAX_VALUE);
        System.out.println("float max:  " + Float.MAX_VALUE);
        System.out.println("double max: " + Double.MAX_VALUE);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
byte:   -128 .. 127
short:  -32768 .. 32767
int:    -2147483648 .. 2147483647
long:   -9223372036854775808 .. 9223372036854775807
char numeric: 0 .. 65535
float max:  3.4028235E38
double max: 1.7976931348623157E308
```

Numeric literals, suffixes and readability · Example 06

The suffix L makes long, F makes float; scientific notation and numeric underscores improve readability.

```
/**
 * CHAPTER 2 | Numeric literals, suffixes and readability
 * WHY IT MATTERS: The suffix L makes long, F makes float; scientific notation and
 * numeric underscores improve readability.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex06Literals {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        long population = 8_200_000_000L; // L mandatory for large long literal
        float temperature = 36.6F;      // without F, 36.6 is double
        double scientific = 6.02e23;
        int binary = 0b1010;            // binary = 10
        int hex = 0xFF;                  // hex = 255
        int octal = 012;                 // octal = 10; avoid leading zeros
        // STEP 2: Compare the output with your prediction.
        System.out.println(population + " " + temperature);
        System.out.println(scientific + " " + binary + " " + hex + " " + octal);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
8200000000 36.6
6.02E23 10 255 10
```

Integer overflow and exact arithmetic · Example 07

Overflow silently wraps around for ordinary integer arithmetic. Math.addExact detects it by throwing ArithmeticException.

```
/**
 * CHAPTER 2 | Integer overflow and exact arithmetic
 * WHY IT MATTERS: Overflow silently wraps around for ordinary integer arithmetic.
 * Math.addExact detects it by throwing ArithmeticException.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex07Overflow {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int max = Integer.MAX_VALUE;
        // STEP 2: Compare the output with your prediction.
        System.out.println(max + 1);    // wraps to Integer.MIN_VALUE
```

```
        try {
            System.out.println(Math.addExact(max, 1));
        } catch (ArithmeticException e) {
            System.out.println("Overflow detected");
        }
        long safer = (long) max + 1;    // promote BEFORE addition
        System.out.println(safer);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
-2147483648
Overflow detected
2147483648
```

Floating-point approximation and special values · Example 08

Never use == for approximate decimal results. Infinity and NaN are valid floating-point values, not exceptions.

```
/**
 * CHAPTER 2 | Floating-point approximation and special values
 * WHY IT MATTERS: Never use == for approximate decimal results. Infinity and NaN are
 * valid floating-point values, not exceptions.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex08FloatingPoint {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        double result = 0.1 + 0.2;
        // STEP 2: Compare the output with your prediction.
        System.out.println(result);     // commonly 0.30000000000000004
        System.out.println(result == 0.3); // false
        System.out.println(Math.abs(result - 0.3) < 1e-9); // tolerance check
        System.out.println(1.0 / 0.0); // Infinity
        System.out.println(0.0 / 0.0); // NaN
        System.out.println(Double.isNaN(0.0 / 0.0));
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
0.30000000000000004
false
true
Infinity
NaN
true
```

char, escapes and Unicode · Example 09

char is one UTF-16 code unit, not necessarily a complete Unicode character; supplementary symbols use two code units.

```
/**
 * CHAPTER 2 | char, escapes and Unicode
 * WHY IT MATTERS: char is one UTF-16 code unit, not necessarily a complete Unicode
 * character; supplementary symbols use two code units.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex09CharUnicode {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        char letter = 'A';
```

```
        char devanagari = '\u0905';   // Unicode escape for Devanagari A
        char newline = '\n';
        // STEP 2: Compare the output with your prediction.
        System.out.println(letter + " numeric code unit " + (int) letter);
        System.out.println(devanagari);
        System.out.print("First line" + newline + "Second line\n");
        String emoji = "";          // often two UTF-16 code units
        System.out.println("Emoji length in UTF-16 units: " + emoji.length());
        System.out.println("Emoji code points: " + emoji.codePointCount(0, emoji.length()));
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
A numeric code unit 65
□
First line
Second line
Emoji length in UTF-16 units: 2
Emoji code points: 1
```

3. Conversions, promotion and casting

Widening converts a narrower compatible numeric type to a wider one automatically; narrowing needs an explicit cast and can lose information. Beware: widening int/long to float/double can lose precision.

Widening vs narrowing · Example 10

A cast tells Java you accept possible truncation/wrap. It does not magically clamp the value into range.

```
/**
 * CHAPTER 2 | Widening vs narrowing
 * WHY IT MATTERS: A cast tells Java you accept possible truncation/wrap. It does not
 * magically clamp the value into range.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex10WideningNarrowing {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int small = 100;
        long wide = small;               // implicit widening
        double decimal = wide;          // implicit widening
        double amount = 12.99;
        int truncated = (int) amount;    // 12, NOT rounded
        int large = 130;
        byte wrapped = (byte) large;     // -126, not 127
        // STEP 2: Compare the output with your prediction.
        System.out.println(wide + " / " + decimal);
        System.out.println(truncated + " / " + wrapped);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
100 / 100.0
12 / -126
```

Arithmetic promotions and integer division · Example 11

byte and short operands normally promote to int. int / int truncates toward zero, even when assigned to double.

```java
/**
 * CHAPTER 2 | Arithmetic promotions and integer division
 * WHY IT MATTERS: byte and short operands normally promote to int. int / int truncates
 * toward zero, even when assigned to double.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex11Promotion {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        byte a = 10;
        byte b = 20;
        int sum = a + b;                // expression promoted to int
        // STEP 2: Compare the output with your prediction.
        System.out.println(sum);
        System.out.println(5 / 2);      // 2
        System.out.println(5 / 2.0);    // 2.5
        double wrong = 5 / 2;           // evaluated as int BEFORE assignment
        double correct = (double) 5 / 2;
        System.out.println(wrong + " vs " + correct);
        System.out.println(-5 / 2);     // -2, truncates toward zero
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
30
2
2.5
2.0 vs 2.5
-2
```

When widening loses precision · Example 12

A very large long cannot always be represented exactly by a double; numeric range and precision are different.

```java
/**
 * CHAPTER 2 | When widening loses precision
 * WHY IT MATTERS: A very large long cannot always be represented exactly by a double;
 * numeric range and precision are different.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex12PrecisionLong {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        long exact = 9_007_199_254_740_993L; // 2^53 + 1
        double approximation = exact;
        // STEP 2: Compare the output with your prediction.
        System.out.println(exact);
        System.out.printf("%.0f%n", approximation);
        System.out.println((long) approximation == exact); // false
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
9007199254740993
9007199254740992
false
```

4. Strings in depth

A String is an immutable sequence of UTF-16 code units. Create it using double quotes; characters use single quotes. String methods typically return NEW values rather than changing the original.

String creation, length and character access · Example 13

Indexes begin at zero; charAt returns a char code unit. Do not treat length() as a human-character count for every Unicode string.

```
/**
 * CHAPTER 2 | String creation, length and character access
 * WHY IT MATTERS: Indexes begin at zero; charAt returns a char code unit. Do not treat
 * length() as a human-character count for every Unicode string.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex13StringBasics {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String firstName = "Java";
        String empty = "";
        // STEP 2: Compare the output with your prediction.
        System.out.println(firstName.length());
        System.out.println(firstName.charAt(0));    // J
        System.out.println(firstName.charAt(firstName.length() - 1)); // a
        System.out.println(empty.isEmpty());
        System.out.println("  ".isBlank());         // Java 11+
        // firstName.charAt(4);                     // runtime exception
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
4
J
a
true
true
```

substring, search, replace and trimming · Example 14

substring(start,end) includes start and excludes end. String methods return a new result.

```
/**
 * CHAPTER 2 | substring, search, replace and trimming
 * WHY IT MATTERS: substring(start,end) includes start and excludes end. String methods
 * return a new result.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex14StringMethods {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String input = "  Learn Java Quickly  ";
        String cleaned = input.strip();             // Unicode-aware Java 11+
        // STEP 2: Compare the output with your prediction.
        System.out.println(cleaned.toUpperCase());
        System.out.println(cleaned.substring(6, 10)); // Java
        System.out.println(cleaned.indexOf("Java"));
        System.out.println(cleaned.contains("Quick"));
        System.out.println(cleaned.replace("Quickly", "Well"));
        System.out.println(input.length());         // input unchanged
```

```
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
LEARN JAVA QUICKLY
Java
6
true
Learn Java Well
22
```

equals vs == and string pool · Example 15

== tests whether references identify the same object; equals checks String content. Interner optimizations can make == appear to work.

```
/**
 * CHAPTER 2 | equals vs == and string pool
 * WHY IT MATTERS: == tests whether references identify the same object; equals checks
 * String content. Interner optimizations can make == appear to work.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex15StringEquality {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String a = "hello";
        String b = "hello";
        String c = new String("hello");
        // STEP 2: Compare the output with your prediction.
        System.out.println(a == b);       // often true: pooled literals
        System.out.println(a == c);       // false: c is a new object
        System.out.println(a.equals(c));  // true: same content
        System.out.println("HELLO".equalsIgnoreCase(a));
        System.out.println("hello".equals(c)); // literal-first null safe
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
true
false
true
true
true
```

null, blank, empty and safe checks · Example 16

null means no object reference. The empty string is an existing String with length zero. isBlank detects whitespace-only text.

```
/**
 * CHAPTER 2 | null, blank, empty and safe checks
 * WHY IT MATTERS: null means no object reference. The empty string is an existing
 * String with length zero. isBlank detects whitespace-only text.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex16StringNull {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String absent = null;
        String empty = "";
        String spaces = "  ";
```

```
        // STEP 2: Compare the output with your prediction.
        System.out.println(absent == null);     // true
        System.out.println(empty.isEmpty());    // true
        System.out.println(spaces.isBlank());   // true
        System.out.println("ok".equals(absent)); // false, safe
        // absent.length();                      // NullPointerException
        if (absent != null && !absent.isBlank()) {
            System.out.println(absent);
        } else {
            System.out.println("No nonblank value");
        }
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
true
true
true
false
No nonblank value
```

Immutability and concatenation · Example 17

toUpperCase and replace cannot change the original String. += builds another String; repeated concatenation in a loop may be inefficient.

```
/**
 * CHAPTER 2 | Immutability and concatenation
 * WHY IT MATTERS: toUpperCase and replace cannot change the original String. += builds
 * another String; repeated concatenation in a loop may be inefficient.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex17StringImmutability {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String name = "java";
        String uppercase = name.toUpperCase();
        // STEP 2: Compare the output with your prediction.
        System.out.println(name);         // still java
        System.out.println(uppercase);    // JAVA
        name += " language";             // name now refers to another String
        System.out.println(name);
        StringBuilder builder = new StringBuilder();
        for (int n = 1; n <= 3; n++) builder.append(n);
        System.out.println(builder.toString());
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
java
JAVA
java language
123
```

Text blocks, formatting and parsing · Example 18

Use formatted and printf for formatting; parseInt parses a numeric String and may throw NumberFormatException. Text blocks need modern Java (15+).

```
/**
 * CHAPTER 2 | Text blocks, formatting and parsing
 * WHY IT MATTERS: Use formatted and printf for formatting; parseInt parses a numeric
```

```
 * String and may throw NumberFormatException. Text blocks need modern Java (15+).
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex18StringFormatting {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String price = "125";
        int value = Integer.parseInt(price);
        System.out.printf("Price: $%,.2f%n", (double) value);
        // STEP 2: Compare the output with your prediction.
        System.out.println("Value=" + value);
        String formatted = "Name: %s | Score: %d".formatted("Maya", 95);
        System.out.println(formatted);
        String message = """
                Java supports
                multiline text blocks.
                """;
        System.out.println(message);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Price: $125.00
Value=125
Name: Maya | Score: 95
Java supports
multiline text blocks.
```

5. Operators and expression evaluation

Operators produce results from operands. Understand type promotion, short-circuiting and precedence instead of relying on memorized rules. Parentheses make intent obvious.

Arithmetic, remainder and negatives · Example 19

% is remainder, not mathematical modulo. Division and remainder with negative operands can surprise learners.

```
/**
 * CHAPTER 2 | Arithmetic, remainder and negatives
 * WHY IT MATTERS: % is remainder, not mathematical modulo. Division and remainder with
 * negative operands can surprise learners.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex19Arithmetic {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int x = 11, y = 4;
        // STEP 2: Compare the output with your prediction.
        System.out.println(x + y);       // 15
        System.out.println(x - y);       // 7
        System.out.println(x * y);       // 44
        System.out.println(x / y);       // 2 (integer division)
        System.out.println(x % y);       // 3
        System.out.println(-11 % 4);     // -3
        System.out.println(Math.floorMod(-11, 4)); // 1
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
15
7
44
2
3
-3
1
```

Prefix/postfix increment and compound assignment · Example 20

++x increments before producing the value; x++ produces the previous value before incrementing. Avoid combining many side effects in one expression.

```java
/**
 * CHAPTER 2 | Prefix/postfix increment and compound assignment
 * WHY IT MATTERS: ++x increments before producing the value; x++ produces the previous
 * value before incrementing. Avoid combining many side effects in one expression.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex20Increment {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int x = 5;
        int before = ++x;               // x = 6, before = 6
        int after = x++;                // after = 6, then x = 7
        // STEP 2: Compare the output with your prediction.
        System.out.println(before + " " + after + " " + x);
        byte small = 10;
        small += 1;                     // implicit cast within compound assignment
        // small = small + 1;           // compile error: int to byte
        System.out.println(small);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
6 6 7
11
```

Logical operators and short circuit · Example 21

&& and || skip the right side when the left determines the result. & and | on booleans evaluate both sides.

```java
/**
 * CHAPTER 2 | Logical operators and short circuit
 * WHY IT MATTERS: && and || skip the right side when the left determines the result. &
 * and | on booleans evaluate both sides.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex21Logical {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String name = null;
        boolean safe = name != null && name.length() > 0;
        // STEP 2: Compare the output with your prediction.
        System.out.println(safe);
        int number = 8;
        System.out.println(number > 0 && number < 10);
        System.out.println(number < 0 || number == 8);
        System.out.println(!(number == 8));
        System.out.println(true ^ false); // boolean exclusive OR
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
false
true
true
false
true
```

Precedence and associativity · Example 22

Multiplication binds more tightly than addition; && binds more tightly than ||. Prefer parentheses in nontrivial expressions.

```
/**
 * CHAPTER 2 | Precedence and associativity
 * WHY IT MATTERS: Multiplication binds more tightly than addition; && binds more
 * tightly than ||. Prefer parentheses in nontrivial expressions.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex22Precedence {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        // STEP 2: Compare the output with your prediction.
        System.out.println(2 + 3 * 4);         // 14
        System.out.println((2 + 3) * 4);       // 20
        boolean result = true || false && false;
        System.out.println(result);            // true
        System.out.println((true || false) && false); // false
        int x = 1;
        int y = 2;
        x = y = 9;                             // assignment right-associative
        System.out.println(x + " " + y);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
14
20
true
false
9 9
```

Conditional ?: operator · Example 23

Use ternary for compact value selection, not side-effect-heavy branching. Type rules can affect the result.

```
/**
 * CHAPTER 2 | Conditional ?: operator
 * WHY IT MATTERS: Use ternary for compact value selection, not side-effect-heavy
 * branching. Type rules can affect the result.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex23Ternary {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int age = 17;
        String category = age >= 18 ? "Adult" : "Minor";
        // STEP 2: Compare the output with your prediction.
        System.out.println(category);
        int score = 86;
        String result = score >= 50 ? "Pass" : "Fail";
        System.out.println(result);
        int a = 3, b = 8;
```

```
        int bigger = a > b ? a : b;
        System.out.println(bigger);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Minor
Pass
8
```

Bitwise shifts, masks and XOR · Example 24

Bitwise operations work on binary integer representations. >> sign-extends; >>> shifts in zeros. Shift distances are masked (int: low 5 bits; long: low 6).

```
/**
 * CHAPTER 2 | Bitwise shifts, masks and XOR
 * WHY IT MATTERS: Bitwise operations work on binary integer representations. >> sign-
 * extends; >>> shifts in zeros. Shift distances are masked (int: low 5 bits; long: low
 * 6).
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex24Bitwise {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int flags = 0b0101;
        int mask = 0b0001;
        // STEP 2: Compare the output with your prediction.
        System.out.println((flags & mask) != 0); // test bit
        System.out.println(Integer.toBinaryString(flags | 0b0010));
        System.out.println(Integer.toBinaryString(flags ^ 0b0001));
        System.out.println(8 << 1);      // 16
        System.out.println(-8 >> 1);     // -4, sign-preserving
        System.out.println(-8 >>> 1);    // large positive number
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
true
111
100
16
-4
2147483644
```

6. Decisions: if, else and switch

Conditions must evaluate to boolean (Java does not treat 1 or 0 as true/false). Branches are tested in order; only the first matching if/else-if branch executes.

if / else-if / else and boundaries · Example 25

Order matters. Check the most selective condition before general ones, and test exact boundary values.

```
/**
 * CHAPTER 2 | if / else-if / else and boundaries
 * WHY IT MATTERS: Order matters. Check the most selective condition before general
 * ones, and test exact boundary values.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
```

```
 */
public class Ex25IfElse {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int score = 85;
        if (score >= 90) {
            // STEP 2: Compare the output with your prediction.
        System.out.println("A");
        } else if (score >= 80) {
            System.out.println("B");
        } else if (score >= 70) {
            System.out.println("C");
        } else {
            System.out.println("Needs work");
        }
        // if (1) { }   // compile error: condition must be boolean
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
B
```

Nested conditions and guard clauses · Example 26

Early guards can keep decision logic readable. The dangling else belongs to the nearest unmatched if.

```
/**
 * CHAPTER 2 | Nested conditions and guard clauses
 * WHY IT MATTERS: Early guards can keep decision logic readable. The dangling else
 * belongs to the nearest unmatched if.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex26NestedIf {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int age = 25;
        boolean hasId = false;
        if (age < 18) {
            // STEP 2: Compare the output with your prediction.
        System.out.println("Too young");
        } else if (!hasId) {
            System.out.println("ID required");
        } else {
            System.out.println("Allowed");
        }
        // Braces eliminate ambiguity around nested if/else.
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
ID required
```

Traditional switch and fall-through · Example 27

Traditional switch supports multiple case labels. Without break, execution falls through to the next case.

```
/**
 * CHAPTER 2 | Traditional switch and fall-through
 * WHY IT MATTERS: Traditional switch supports multiple case labels. Without break,
 * execution falls through to the next case.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex27SwitchStatement {
```

```
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int day = 6;
        switch (day) {
            case 1: case 2: case 3: case 4: case 5:
                // STEP 2: Compare the output with your prediction.
        System.out.println("Weekday");
                break;
            case 6: case 7:
                System.out.println("Weekend");
                break;
            default:
                System.out.println("Invalid day");
        }
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Weekend
```

Modern switch expression with arrows · Example 28

Arrow cases do not fall through. Switch expressions yield a value and must cover all possible inputs (typically with default).

```
/**
 * CHAPTER 2 | Modern switch expression with arrows
 * WHY IT MATTERS: Arrow cases do not fall through. Switch expressions yield a value
 * and must cover all possible inputs (typically with default).
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex28SwitchExpression {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String season = "WINTER";
        String label = switch (season) {
            case "WINTER" -> "Cold";
            case "SUMMER" -> "Hot";
            case "SPRING", "FALL" -> "Mild";
            default -> "Unknown";
        };
        // STEP 2: Compare the output with your prediction.
        System.out.println(label);
        int number = 3;
        String kind = switch (number) {
            case 0 -> "zero";
            default -> { yield number > 0 ? "positive" : "negative"; }
        };
        System.out.println(kind);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Cold
positive
```

7. Loops: repetition and control

A loop repeats work while a condition holds. Choose for when iterations are naturally counted, while when repetitions are condition-driven, and do-while when the body must run at least once.

Counted for loop and off-by-one · Example 29

for has initialization, condition and update. The common pattern i < limit runs limit times starting from 0.

```
/**
 * CHAPTER 2 | Counted for loop and off-by-one
 * WHY IT MATTERS: for has initialization, condition and update. The common pattern i <
 * limit runs limit times starting from 0.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex29For {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        for (int i = 0; i < 5; i++) {
            System.out.print(i + " ");   // 0 1 2 3 4
        }
        // STEP 2: Compare the output with your prediction.
        System.out.println();
        for (int i = 5; i >= 1; i--) {
            System.out.print(i + " ");
        }
        System.out.println();
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
0 1 2 3 4
5 4 3 2 1
```

while loop with explicit progress · Example 30

The condition is evaluated BEFORE each iteration. Forgetting to update the variable can cause an infinite loop.

```
/**
 * CHAPTER 2 | while loop with explicit progress
 * WHY IT MATTERS: The condition is evaluated BEFORE each iteration. Forgetting to
 * update the variable can cause an infinite loop.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex30While {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int count = 3;
        while (count > 0) {
            // STEP 2: Compare the output with your prediction.
        System.out.println("Countdown: " + count);
            count--;                    // termination progress
        }
        System.out.println("Go!");
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Countdown: 3
Countdown: 2
Countdown: 1
Go!
```

do-while executes at least once · Example 31

The condition is tested AFTER the body, so the body executes once even if the condition is initially false.

```java
/**
 * CHAPTER 2 | do-while executes at least once
 * WHY IT MATTERS: The condition is tested AFTER the body, so the body executes once
 * even if the condition is initially false.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex31DoWhile {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int count = 0;
        do {
            // STEP 2: Compare the output with your prediction.
        System.out.println("Runs at least once: " + count);
            count++;
        } while (count < 0);
        System.out.println("Done");
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Runs at least once: 0
Done
```

break exits; continue skips to next iteration · Example 32

break ends the nearest loop; continue proceeds to the next iteration. Use sparingly to keep control flow understandable.

```java
/**
 * CHAPTER 2 | break exits; continue skips to next iteration
 * WHY IT MATTERS: break ends the nearest loop; continue proceeds to the next
 * iteration. Use sparingly to keep control flow understandable.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex32BreakContinue {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        for (int n = 1; n <= 10; n++) {
            if (n == 3) continue;      // skip 3
            if (n == 8) break;         // stop before 8
            System.out.print(n + " ");
        }
        // STEP 2: Compare the output with your prediction.
        System.out.println();
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
1 2 4 5 6 7
```

Nested loops and loop variables · Example 33

For each outer iteration, the inner loop runs completely. Total work grows multiplicatively.

```java
/**
 * CHAPTER 2 | Nested loops and loop variables
 * WHY IT MATTERS: For each outer iteration, the inner loop runs completely. Total work
 * grows multiplicatively.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex33NestedLoops {
```

```
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        for (int row = 1; row <= 3; row++) {
            for (int col = 1; col <= 4; col++) {
                System.out.print(row * col + "\t");
            }
            // STEP 2: Compare the output with your prediction.
        System.out.println();
        }
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
1□2□3□4□
2□4□6□8□
3□6□9□12□
```

Labeled break and continue · Example 34

Labels can escape nested loops, but usually a refactoring is clearer. Label names are not variables.

```
/**
 * CHAPTER 2 | Labeled break and continue
 * WHY IT MATTERS: Labels can escape nested loops, but usually a refactoring is
 * clearer. Label names are not variables.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex34LabeledBreak {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        outer:
        for (int row = 1; row <= 3; row++) {
            for (int col = 1; col <= 3; col++) {
                if (row == 2 && col == 2) break outer;
                // STEP 2: Compare the output with your prediction.
        System.out.println("row=" + row + ", col=" + col);
            }
        }
        System.out.println("Exited both loops");
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
row=1, col=1
row=1, col=2
row=1, col=3
row=2, col=1
Exited both loops
```

Three common loop traps · Example 35

Watch boundary conditions, integer arithmetic, and values that fail to make progress. Use long if the counter could overflow.

```
/**
 * CHAPTER 2 | Three common loop traps
 * WHY IT MATTERS: Watch boundary conditions, integer arithmetic, and values that fail
 * to make progress. Use long if the counter could overflow.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex35LoopPitfalls {
    public static void main(String[] args) {
```

```
        // STEP 1: Follow each statement in order; watch how values and types change.
        int sum = 0;
        for (int i = 1; i <= 5; i++) sum += i;
        // STEP 2: Compare the output with your prediction.
        System.out.println("1..5 sum: " + sum); // 15
        int i = 0;
        while (i < 3) {
            System.out.println("step=" + i);
            i++;                        // without increment, infinite loop
        }
        // for (int k = 0; k <= 5; k++) has 6 iterations, not 5.
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
1..5 sum: 15
step=0
step=1
step=2
```

8. Synthesis: predicting output and fixing bugs

Reading unfamiliar code is a core refresher skill. Trace values in order, track expression types, and identify whether the issue occurs at compile time or runtime.

Trace mixed expressions · Example 36

Predict the printed value before running: follow prefix/postfix order and integer division.

```
/**
 * CHAPTER 2 | Trace mixed expressions
 * WHY IT MATTERS: Predict the printed value before running: follow prefix/postfix
 * order and integer division.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex36TraceExpressions {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        int a = 4;
        int b = a++ + 2;             // b = 6, a = 5
        int c = ++a * 2;            // a = 6, c = 12
        double d = a / 4;           // int division: 6 / 4 = 1
        // STEP 2: Compare the output with your prediction.
        System.out.println("a=" + a + ", b=" + b + ", c=" + c + ", d=" + d);
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
a=6, b=6, c=12, d=1.0
```

Corrected examples of frequent mistakes · Example 37

Compare correct expressions to common errors in comments. Syntax, equality, null, and division are frequent sources of bugs.

```
/**
 * CHAPTER 2 | Corrected examples of frequent mistakes
 * WHY IT MATTERS: Compare correct expressions to common errors in comments. Syntax,
 * equality, null, and division are frequent sources of bugs.
 * QUICK PRACTICE: Predict printed values before running.
```

```
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex37FindTheBug {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        String name = "Java";
        if ("Java".equals(name)) System.out.println("Matches");
        // if (name == "Java") ... compares references instead of content
        int total = 5;
        int count = 2;
        double average = (double) total / count;
        // double wrong = total / count; // 2.0, integer division first
        // STEP 2: Compare the output with your prediction.
        System.out.println(average);
        String optional = null;
        if (optional != null && !optional.isBlank()) {
            System.out.println(optional);
        }
        // if (optional.isBlank()) throws NullPointerException
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
Matches
2.5
```

Exact decimal arithmetic with BigDecimal · Example 38

For money, construct BigDecimal from decimal String, not a floating-point double; this avoids importing a pre-existing binary approximation.

```
/**
 * CHAPTER 2 | Exact decimal arithmetic with BigDecimal
 * WHY IT MATTERS: For money, construct BigDecimal from decimal String, not a floating-
 * point double; this avoids importing a pre-existing binary approximation.
 * QUICK PRACTICE: Predict printed values before running.
 * Run directly in IntelliJ IDEA (JDK 17+).
 */
public class Ex38BigDecimalMoney {
    public static void main(String[] args) {
        // STEP 1: Follow each statement in order; watch how values and types change.
        java.math.BigDecimal price = new java.math.BigDecimal("0.10");
        java.math.BigDecimal tax = new java.math.BigDecimal("0.20");
        // STEP 2: Compare the output with your prediction.
        System.out.println(price.add(tax));  // exactly 0.30
        System.out.println(0.1 + 0.2);      // binary float approximation
    }
}
```

EXPECTED OUTPUT (verified by running the example)

```
0.30
0.30000000000000004
```

9. Mastery checks and reference

Use these as a short closed-book review before moving to the next Java chapter.

> Predict output Q1 What does `double x = 7 / 2;` store? Q2 What is `-7 % 3`? Q3 Does `"abc" == new String("abc")` return true? Q4 What does `for (int i=0;i<=3;i++)` run: 3 or 4 iterations? Q5 Why is `String s=null; s.length()` unsafe?

Answers A1 3.0, because integer division happens first. A2 -1. A3 false (use equals for contents). A4 four. A5 null is not an object; method invocation raises NullPointerException.

Compile-error challenge Explain why each fails: `byte b=128;` (out of range); `float f=1.5;` (double literal); `var x;` (no initializer); `if(1)` (not boolean); `int z=4.7;` (lossy conversion); `int a=1; int a=2;` in the same scope (duplicate declaration).

Operator precedence pocket reference High to low (simplified): postfix x++; unary ++x, !, ~, cast; multiplicative * / %; additive + -; shifts << >> >>>; relational < <= > >= instanceof; equality == !=; bitwise & ^ |; logical && ||; ternary ?:; assignment = += .... Use parentheses when in doubt.

Java version and next steps All examples target Java 17+ (also compatible with modern Java LTS versions). The runner methods are boilerplate needed to run files independently; classes, methods, arrays, reference types beyond String, and deeper OOP are intentionally deferred. For each file in IntelliJ: open project, set JDK 17+, right-click the file or green run arrow, and run its main method.

Simple Constructs Big Power
Master these fundamentals!

Java Language Refresher – Chapter 2

Variables • Primitive Types • Operators • Conditions • Loops

Key Building Blocks for Spring Boot and Advanced Java

1 Variables & Scope

Store values and control visibility



- Declaration, initialization, assignment
- Meaningful names (lowerCamelCase)
- final and var keywords
- Scope: local, block, method
- Variables must be assigned before use

```
int score = 10;   // declaration + init
int bonus = 3;
score = score + bonus;  // 13
final double PI = 3.14;
var name = "Java";
```

Scope Examples

- Method scope (visible inside method)
- Block scope (visible inside { } only)
- Cannot use variable outside its scope

Common Mistakes

- Using variable before initialization
- Accessing variable outside its scope

2 Primitive Types

8 built-in types with fixed ranges

Type	Size	Range (Approximate)
byte	8 bits	-128 to 127
short	16 bits	-32,768 to 32,767
int	32 bits	-2.1 billion to 2.1 billion
long	64 bits	-9.2e18 to 9.2e18
float	32 bits	~6-7 decimal digits
double	64 bits	~15-16 decimal digits
char	16 bits	0 to 65,535 (Unicode)
boolean	varies	true or false

3 Operators

Perform operations on values

Category	Operators	Example	Result
Arithmetic	+ - * / %	7 % 3	1
Assignment	= += -= *= /=	x *= 5	x = x + 5
Comparison	== != > < >= <=	5 == 5	true
Logical	&& | !	true && false	false
Bitwise	& | ^ ~ << >>	5 << 1	10
Unary	+ - ++ -- !	++x	x = x + 1
Ternary	? :	age > 18 ? "Adult" : "Minor"	

Quick Examples

```
int a = 10, b = 3;
System.out.println(a + b);   // 13
System.out.println(a % b);   // 1
System.out.println(a > b);   // true
```

Operator Precedence (High → Low)

() → unary → * / % → + - → << >> → < <= > >= → == != → & → ^ → | → && → || → ? : → = += -= ...

4 Type Casting & Conversion

Change a value from one type to another

Automatic Widening (safe)

double
float
long
int
short
byte

Explicit Narrowing (cast needed)

Examples

```
int i = 100;
long l = i;          // widening (auto)
double d = i;        // widening (auto)

int n = (int) 9.7;   // narrowing (cast)
byte b = (byte) 130; // 130 → -126 (overflow)
```

Important Pionts

- Widening is automatic and safe.
- Narrowing needs an explicit cast.
- int arithmetic promotes to int.
- Be careful with overflow when narrowing.

5 Strings

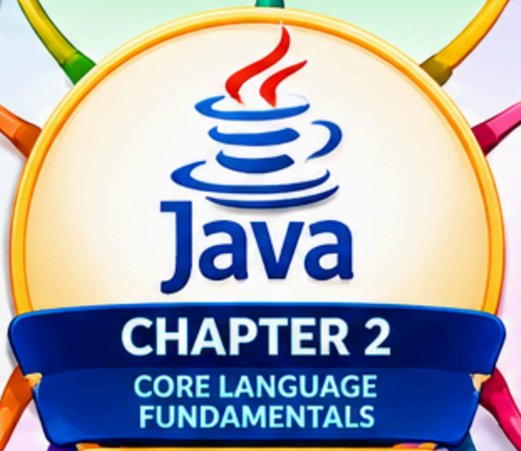
Work with text (not a primitive)

```
String s = "Java";
int len = s.length();              // 4
char ch = s.charAt(1);             // 'a'
String upper = s.toUpperCase();    // "JAVA"
String sub = s.substring(1, 3);    // "av"
boolean eq = s.equals("Java");     // true
```

Common String Operations

- length() – number of characters
- charAt(i) – character at index
- substring(start, end) – part of string
- equals() – compare content
- + – concatenation
- toUpperCase(), toLowerCase()

String is immutable
Methods create new strings. Original string does not change.



6 Conditions & Switch

Make decisions in your code

if – else

```
int age = 18;
if (age >= 18) {
    System.out.println("Adult");
} else {
    System.out.println("Minor");
}
```

Ternary Operator

```
String type = (age >= 18) ?
              "Adult" : "Minor";
```

switch (Java 17)

```
switch (day) {
    case 1 -> System.out.println("Mon");
    case 2,3,4,5 -> System.out.println("Weekday");
    default -> System.out.println("Weekend");
}
```

When to use?

- if – simple conditions
- else if – multiple conditions
- switch – many fixed values
- ternary – short expression

Common Mistakes

- Missing break in switch (pre Java 14)
- Using = instead of ==
- Incorrect condition logic

7 Loops

Repeat a block of code

for loop

```
for (int i = 1; i <= 3; i++) {
    System.out.println(i);
}
```

Output:

```
1
2
3
```

while loop

```
int i = 1;
while (i <= 3) {
    System.out.println(i);
    i++;
}
```

Output:

```
1
2
3
```

do-while loop

```
int i = 1;
do {
    System.out.println(i);
    i++;
} while (i <= 3);
```

Output:

```
1
2
3
```

Loop Controls

break	- exit loop
continue	- skip to next iteration
nested loops	- loop inside loop

Common Mistakes

- Infinite loop (condition never false)
- Off-by-one errors
- Forgetting to update counter
- Misusing break/continue

8 Quick Examples & Output

See it all together

```
int age = 46;
double price = 19.95;
boolean isMember = true;
char grade = 'A';
System.out.println(age + " | " + price + " | " + isMember + " | " + grade);
System.out.println("Next age: " + (age + 1));
```

Output:

```
46 | 19.95 | true | A
Next age: 47
```

Key Takeaways

- Choose the right type for the right value.
- Understand operator precedence.
- Use meaningful variable names.
- Write clear and readable conditions and loops.
- Avoid common mistakes.